IsabelleGym Server API Documentation

Version 0.0.1

Overview

Title: IsabelleGym Server
Description: RESTful API for Isabelle theorem proving.
OpenAPI version: 3.1.0
API version: 0.0.1
This API exposes session lifecycle operations, theorem-proving commands, proof-state inspection, checkpoints, rollback, history, and a one-shot "big step" execution path.

Session Lease Model

Several session-specific endpoints accept an optional header named `X-Lease-Id`. In practice, this acts as an exclusive lease token for a session.
The lease workflow is:

1. Create or acquire a session.
2. Store the returned `session_id` and `lease_id`.
3. Pass `X-Lease-Id: <lease_id>` to session-specific endpoints.
4. Release the session with `POST /api/v1/sessions/{session_id}/release` when finished, or delete it with `DELETE /api/v1/sessions/{session_id}`.

The `/api/v1/sessions/acquire` endpoint explicitly guarantees exclusive leasing until the caller releases or deletes the session.

Endpoints

Root

GET /

- Summary: Root
- Response 200: Successful response, schema unspecified.

Sessions Collection

GET /api/v1/sessions

- Summary: List Sessions
- Response 200: Successful response, schema unspecified.

POST `/api/v1/sessions`

- Summary: Create Session
- Request body: `SessionCreateRequest` or `null`
- Response 200: `SessionResponse`
- Response 422: `HTTPValidationError`

Acquire Session

POST `/api/v1/sessions/acquire`

- Summary: Acquire Session
- Description: Finds an existing session that matches requested dependencies and field, or creates one on demand. The returned session is exclusively leased and will not be handed out again until released or deleted.
- Request body: required, `SessionAcquireRequest`
- Response 200: `SessionAcquireResponse`
- Response 422: `HTTPValidationError`

Release Session

POST `/api/v1/sessions/{session_id}/release`

- Summary: Release Session
- Description: Releases the exclusive lease on a session and returns it to the pool for reuse. Unlike delete, the backend stays alive.
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response 200: Successful response, schema unspecified.
- Response 422: `HTTPValidationError`

Session Info and Close

GET `/api/v1/sessions/{session_id}`

- Summary: Get Session Info
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional

- Response 200: Successful response, schema unspecified.
- Response 422: `HTTPValidationError`

DELETE `/api/v1/sessions/{session_id}`

- Summary: Close Session
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response 200: Successful response, schema unspecified.
- Response 422: `HTTPValidationError`

Execute Command

POST `/api/v1/sessions/{session_id}/commands`

- Summary: Execute Command
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Request body: required, `CommandRequest`
- Response 200: `CommandResponse`
- Response 422: `HTTPValidationError`

Proof State

GET `/api/v1/sessions/{session_id}/state`

- Summary: Get Proof State
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response 200: `ProofStateResponse`
- Response 422: `HTTPValidationError`

Subgoals

GET `/api/v1/sessions/{session_id}/subgoals`

- Summary: Get Subgoals
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Source

GET `/api/v1/sessions/{session_id}/source`

- Summary: Get Source
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Checkpoints

POST `/api/v1/sessions/{session_id}/checkpoints`

- Summary: Save Checkpoint
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: `StateCheckpoint`
- Response `422`: `HTTPValidationError`

POST `/api/v1/sessions/{session_id}/checkpoints/{checkpoint_id}/restore`

- Summary: Restore Checkpoint
- Path parameters:
  - `session_id`: string, required

  - `checkpoint_id`: integer, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Rollback

POST `/api/v1/sessions/{session_id}/rollback`

- Summary: Rollback
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Enter Theory

POST `/api/v1/sessions/{session_id}/enter_theory/{theory_name}`

- Summary: Enter Theory
- Path parameters:
  - `session_id`: string, required
  - `theory_name`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Command History

GET `/api/v1/sessions/{session_id}/history`

- Summary: Get Command History
- Path parameters:
  - `session_id`: string, required
- Query parameters:
  - `limit`: integer, optional, default `50`

- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Big Step

POST `/api/v1/sessions/bigstep`

- Summary: Execute Big Step
- Request body: required, `BigStepTheoryRequest`
- Response `200`: `CommandResponse`
- Response `422`: `HTTPValidationError`

Session Statistics

GET `/api/v1/sessions/{session_id}/stats`

- Summary: Get Session Stats
- Path parameters:
  - `session_id`: string, required
- Header parameters:
  - `X-Lease-Id`: string or null, optional
- Response `200`: Successful response, schema unspecified.
- Response `422`: `HTTPValidationError`

Schemas

BigStepTheoryRequest

Field	Type	Required	Notes
`theory_name`	string	yes	Theory name.
`dependencies`	array[string]	no	Dependency list.
`field`	string or null	no	Optional field selector.
`theory`	string	yes	Theory content.
`timeout`	number	no	Default: `300`.

Field	Type	Required	Notes

CommandRequest

Field	Type	Required	Notes
`command`	string	yes	Command to execute.
`timeout`	number or null	no	Default: `30`.

CommandResponse

Field	Type	Required	Notes
`success`	boolean	yes	Whether command execution succeeded.
`output`	string or null	no	Command output.
`error`	string or null	no	Error message.
`subgoal_error`	string or null	no	Subgoal parsing or retrieval error.
`subgoals`	array[string]	yes	Current subgoals.
`execution_time`	number	yes	Execution time.
`mode`	string or null	no	Optional execution mode.
`diagnostics`	array[Any]	no	Diagnostics list.
`failure_location`	FailureLocationResponse or null	no	Failure location metadata.
`theory_verified`	boolean	no	Default: `false`.

FailureLocationResponse

Field	Type	Required	Notes
`block_index`	integer	yes	Index of failing block.
`chunk_index`	integer or null	no	Index of failing chunk.
`preview`	string or null	no	Optional preview text.

HTTPValidationError

Field	Type	Required	Notes
`detail`	array[ValidationError]	no	Validation error details.

ProofStateResponse

Field	Type	Required	Notes
`subgoals`	array[string]	yes	Current subgoals.
`proof_finished`	boolean	yes	Whether proof is complete.

Field	Type	Required	Notes
`current_theory`	string	yes	Active theory name.

SessionAcquireRequest

Field	Type	Required	Notes
`theories`	array[string]	no	Requested theories.
`field`	string or null	no	Optional field selector.
`reuse_dirty`	boolean	no	Default: `true`. If true, reuse sessions with existing command history; if false, only reuse clean sessions.

SessionAcquireResponse

Field	Type	Required	Notes
`session_id`	string	yes	Session identifier.
`created_at`	number	yes	Creation timestamp.
`theories`	array[string]	yes	Loaded theories.
`status`	string	yes	Session status.
`reused`	boolean	yes	True if an existing session was returned.
`lease_id`	string	yes	Exclusive lease identifier. Pass to release endpoint.

SessionCreateRequest

Field	Type	Required	Notes
`theories`	array[string] or null	no	Optional initial theories.
`field`	string or null	no	Optional field selector.

SessionResponse

Field	Type	Required	Notes
`session_id`	string	yes	Session identifier.
`created_at`	number	yes	Creation timestamp.
`theories`	array[string]	yes	Loaded theories.
`status`	string	yes	Session status.
`lease_id`	string	yes	Exclusive lease identifier required for session-specific endpoints.

StateCheckpoint

Field	Type	Required	Notes
`checkpoint_id`	integer	yes	Checkpoint identifier.
`timestamp`	number	yes	Checkpoint timestamp.

ValidationError

Field	Type	Required	Notes
`loc`	array[string or integer]	yes	Error location path.
`msg`	string	yes	Human-readable message.
`type`	string	yes	Error type.
`input`	Any	no	Input value associated with error.
`ctx`	object	no	Optional context.

Typical Workflow

1. Acquire a leased session:

```
POST /api/v1/sessions/acquire
{
  "theories": ["Main"],
  "field": null,
  "reuse_dirty": true
}
```

2. Read `session_id` and `lease_id` from the response.

3. Execute commands against that session, sending the lease header:

```
POST /api/v1/sessions/{session_id}/commands
X-Lease-Id: {lease_id}

{
  "command": "lemma foo: True by simp",
  "timeout": 30
}
```

4. Inspect state with `GET /api/v1/sessions/{session_id}/state`.

5. Optionally save a checkpoint and restore or roll back later.

6. Release the session when done:

```
POST /api/v1/sessions/{session_id}/release
X-Lease-Id: {lease_id}
```